

VO Computernetzwerke

Kapitel 4: Tunnelling

Ausführliches Skript zum Foliensatz

Sommersemester 2026

Lehrender: Armin Veichtlbauer

Hinweis zur Erstellung: Das Skript enthält die vollständigen Aussagen des Foliensatzes einschließlich der sichtbaren handschriftlichen Ergänzungen. Unklare oder knappe Stellen wurden durch sachliche Erklärungen ergänzt. Die eingefügten Abbildungen sind direkte Ausschnitte aus den Folien.

Quelle: CNW-2026S-Kapitel4.pdf, 21 Folien.

Inhaltsübersicht

1. Überblick und Lernziele
2. Tunneling-Grundlagen
 - 2.1 Netzwerk-Stacks im Internet
 - 2.2 Idee und Ablauf des Tunneling
 - 2.3 6in4 und verwandte Varianten
 - 2.4 Modbus/TCP als Enkapsulierungsbeispiel
3. Secure Shell (SSH)
 - 3.1 Aufgaben und Anwendungsfälle
 - 3.2 Telnet als unsichere Ausgangslösung
 - 3.3 Aufbau einer SSH-Sitzung
 - 3.4 SSH-Tunneling und Paketstruktur
 - 3.5 Port Forwarding
4. Virtual Private Networks (VPN)
 - 4.1 Grundidee, Site-to-Site und Remote Access
 - 4.2 VPN-Protokolle
 - 4.3 Schicht-2-VPNs
 - 4.4 SSL/TLS-VPNs
 - 4.5 Split Tunnel Routing
5. Zusammenfassung der Kernbegriffe

1. Überblick und Lernziele

Kapitel 4 behandelt Tunnelling, Secure Shell und Virtual Private Networks. Die Leitfragen des Foliensatzes sind:

Grundlagen

- Was ist Tunnelling?
- Wie kann Tunnelling in einem Netzwerk-Stack abgebildet werden?
- Wie funktioniert die Beispielanwendung 6in4?

SSH

- Was ist SSH?
- Wie funktioniert ein SSH-Tunnel?
- Was kann mit einem SSH-Tunnel typischerweise gemacht werden?
- Wie sieht der Netzwerk-Stack bei SSH-Tunnelling aus?

VPN

- Wie funktioniert ein VPN?
- Wie kann ein VPN mit Techniken des Tunnellings realisiert werden?
- Welche Anwendungen gibt es dafür?

Die folgenden Abschnitte beantworten diese Fragen systematisch und folgen der Reihenfolge des Foliensatzes.

2. Tunneling-Grundlagen

2.1 Vorbemerkungen: Netzwerk-Stacks im Internet

Beispiel: Drei Netze werden durch zwei Router verbunden. Ein Frame vom Client zum Server durchläuft alle drei Netze: Client → Router R1, R1 → Router R2 und R2 → Server. Damit entstehen drei Hops.

- Das IP-Paket bleibt auf dem Weg grundsätzlich gleich.
- Ethernet-Header und Ethernet-Trailer sind in jedem Netz beziehungsweise auf jedem Hop verschieden.
- Die Daten aus höheren Schichten werden von IP übernommen; IP fügt seinen Header vor den Nutzdaten hinzu.
- Im IP-Header stehen unter anderem Quell- und Zieladresse. Die Foliennotiz nennt außerdem eine Checksumme am Ende.
- Der Client prüft, ob sich das Ziel im eigenen IP-Netz befindet. Liegt das Ziel im selben Netz, wird die Ziel-MAC-Adresse in den Ethernet-Header eingetragen. Liegt es in einem anderen Netz, wird als Ethernet-Zieladresse die MAC-Adresse des Default Gateways verwendet. An jedem Router wird der alte Ethernet-Rahmen entfernt und für den nächsten Hop ein neuer Rahmen erzeugt.

Sachliche Ergänzung: Ein Hop ist ein einzelner Weiterleitungsschritt zwischen zwei direkt verbundenen Netzwerkknoten. Der IPv4-Header besitzt eine Header-Prüfsumme; IPv6 besitzt keine Header-Prüfsumme. Ethernet kann zusätzlich einen Frame Check Sequence-Wert im Trailer enthalten. Die Quell- und Ziel-IP-Adressen bleiben bei normalem Routing erhalten, während die MAC-Adressen pro Hop wechseln.

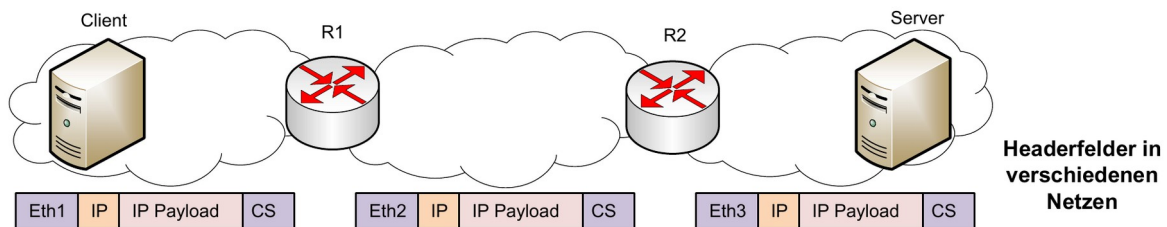


Abbildung 1: Netzwerk-Stacks über drei Netze. Das IP-Paket bleibt erhalten; die Ethernet-Header und -Trailer werden pro Hop neu gebildet. Direkter Ausschnitt aus Folie 3.

2.2 Idee von Tunneling

Daten werden aus verschiedenen Gründen durch einen Tunnel geschickt:

- Absicherung von Daten, die ohne Tunnel ungesichert wären.
- Traversierung blockierender Elemente, zum Beispiel Firewalls.
- Traversierung von Netzbereichen, die bestimmte Protokolle nicht unterstützen, zum Beispiel wenn keine durchgängige IPv6-Strecke verfügbar ist.

Die zu transportierenden Daten können als Ganzes enkapsuliert und durch den Tunnel versendet werden. Vor der Enkapsulierung können sie zusätzlich verschlüsselt werden.

- Im Tunnel werden eigene äußere Header hinzugefügt. Ohne passende Header kann ein Paket nicht über das Transportnetz gesendet und geroutet werden.
- Die Originaldaten müssen im Tunnel nicht erkennbar sein, weil sie als Nutzlast des äußeren Protokolls transportiert werden.

Begriff: Enkapsulierung bedeutet, dass ein vollständiges Protokolldatenelement als Nutzlast in ein anderes Protokolldatenelement eingefügt wird. Dekapsulierung ist das Entfernen der äußeren Header und gegebenenfalls Trailer am Tunnelende.

Ablauf am Tunnelanfang und Tunnelende

- Der Tunnelstart enkapsuliert die Daten, die vom Client kommen.
- Das Tunnelende dekapsuliert diese Daten.
- Im Tunnel sind Header „doppelt“ vorhanden: außen die Header für den Transport durch das Tunnelnetz und innen die ursprünglichen Header der Client-Verbindung.
- In der schematischen Darstellung wird kein Ethernet-Header innerhalb des End-to-End-Tunnels gezeigt, weil der Tunnel mehrere Router durchlaufen kann. Die Ethernet-Header würden an jedem Router ohnehin entfernt und ersetzt.
- Die Verbindung vor dem Tunnelstart und nach dem Tunnelende ist nicht automatisch verschlüsselt. Diese Bereiche sollten deshalb vertrauenswürdig sein. Am sichersten ist ein Aufbau, bei dem der Tunnel direkt am Client beginnt und möglichst nahe am eigentlichen Ziel endet.
- Bei einem SSH-Tunnel ist der innere Datenteil verschlüsselt. Tunnelstart und Tunnelende haben beim Verbindungsaufbau Schlüssel- und Sitzungsdaten ausgetauscht und können die ankommenden SSH-Daten daher korrekt zuordnen und entschlüsseln.
- Die Foliennotez nennt den Diffie-Hellman-Schlüsselaustausch als Verfahren für den initialen Schlüsselaustausch.

Präzisierung: Die inneren Nutzdaten sind bei einem korrekt aufgebauten SSH-Tunnel nicht lesbar. Äußere IP- und TCP-Header bleiben jedoch sichtbar. Je nach Port, Protokollkennung und Handshake kann der Verkehr weiterhin als SSH-Verkehr erkannt werden; verborgen ist vor allem der Inhalt der getunnelten Verbindung.

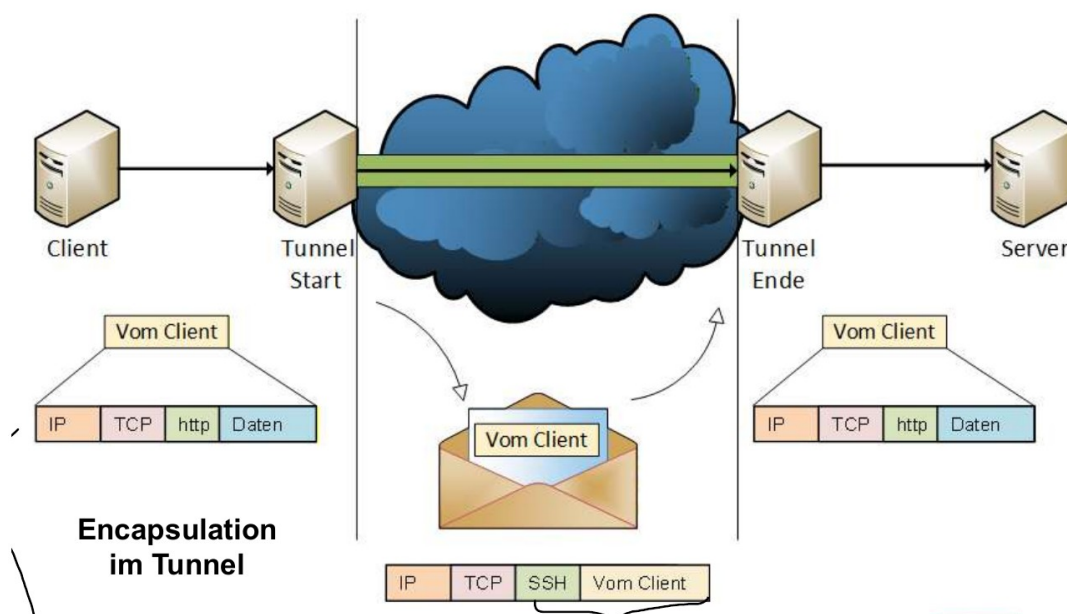


Abbildung 2: Enkapsulierung zwischen Tunnelstart und Tunnelende. Die ursprüngliche Client-Nachricht wird als Nutzlast einer äußeren Verbindung transportiert. Direkter Ausschnitt aus Folie 5.

2.3 Beispiel: 6in4

Ausgangsproblem: Ein Sender möchte IPv6-Pakete an einen Empfänger senden, aber entlang aller möglichen Routen ist kein durchgängiges IPv6 verfügbar, weil IPv6 in einem Zwischenbereich nicht konfiguriert ist.

Lösung: Die IPv6-Pakete werden durch ein IPv4-Netz getunnelt.

- Das vollständige IPv6-Paket wird in ein IPv4-Paket eingepackt.
- Das IPv4-Paket wird zum IPv4-Tunnelende geroutet.
- Am Tunnelende wird der äußere IPv4-Header entfernt; das ursprüngliche IPv6-Paket wird weitergesendet.
- Beide Tunnelenden müssen Dual Stack unterstützen, also IPv4 und IPv6 verstehen. Am Eingang erfolgt der Wechsel vom IPv6-Stack zum IPv4-Transport; am Ausgang erfolgt der Wechsel zurück zu IPv6.
- Im IPv4-Bereich enthält das Paket zwei IP-Header: außen IPv4 für den Tunneltransport und innen IPv6 für die ursprüngliche Ende-zu-Ende-Kommunikation.
- 6in4 verschlüsselt die Daten nicht automatisch. Es verpackt sie lediglich in ein anderes IP-Protokoll.

Sachliche Ergänzung: Bei klassischem 6in4 wird IPv6 direkt als Nutzlast von IPv4 transportiert. Dafür wird im IPv4-Header die IP-Protokollnummer 41 verwendet. Da ein zusätzlicher IPv4-Header entsteht, sinkt die verfügbare Nutzlastgröße gegenüber einer direkten IPv6-Verbindung.

Varianten

- Bei 6to4-Tunneling können die benötigten IPv4-Adressen automatisch aus der IPv6-Adresse extrahiert werden.
- Umgekehrt können auch IPv4-Pakete durch IPv6-Netze getunnelt werden.

Einordnung: 6in4 bezeichnet im Allgemeinen einen konfigurierten IPv6-in-IPv4-Tunnel. 6to4 ist ein automatisches, historisches Übergangsverfahren mit einer aus der IPv4-Adresse abgeleiteten IPv6-Präfixstruktur. Beide Verfahren stellen ohne zusätzliche Sicherheitsmechanismen keine Verschlüsselung bereit.

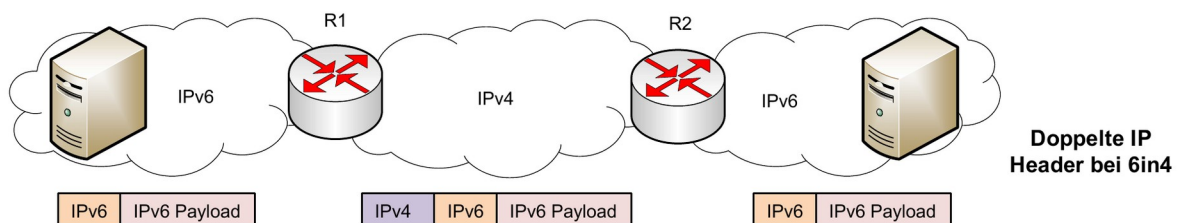


Abbildung 3: 6in4 mit doppeltem IP-Header im IPv4-Zwischennetz. Direkter Ausschnitt aus Folie 7.

2.4 Beispiel: Modbus/TCP

Problem: Sensoren im Feld sollen Daten an einen Client senden, der nicht direkt verbunden ist. Die Kommunikation muss deshalb über IP-Netze geroutet werden.

- Modbus RTU, die ursprüngliche Modbus-Variante, besitzt als Adresse nur eine lokale Slave ID.
- Wenn zwischen den Netzen von Client und Server eine TCP-Verbindung möglich ist, kann der Modbus-Inhalt in ein TCP-Segment enkapsuliert werden.

Die Abbildung vergleicht den Aufbau von Modbus RTU und Modbus TCP:

- Modbus RTU enthält Slave ID, Function Code, Data und CRC.
- Modbus TCP verwendet einen MBAP-Header mit Transaction ID, Protocol ID, Length und Unit ID.
- Die Modbus-PDU besteht bei Modbus TCP aus Function Code und Data.

Sachliche Ergänzung: Die Transaction ID ordnet Antworten den zugehörigen Anfragen zu. Protocol ID kennzeichnet das verwendete Protokoll und ist bei normalem Modbus TCP typischerweise 0. Length beschreibt die Länge der nachfolgenden Felder. Unit ID dient insbesondere beim Übergang über Gateways zur Adressierung eines nachgelagerten Geräts. Ein eigener Modbus-RTU-CRC wird im Modbus-TCP-Datenfeld nicht mitgeführt, weil die unteren Netzwerkschichten bereits Fehlererkennungsmechanismen bereitstellen.

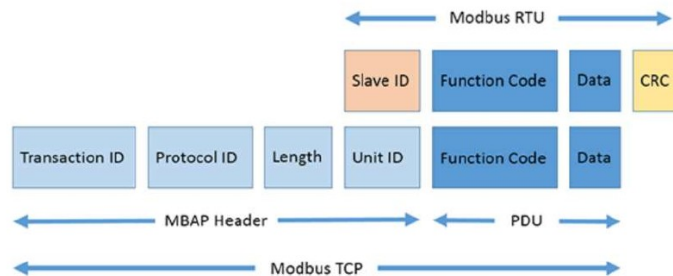


Abbildung 4: Aufbau von Modbus RTU und Modbus TCP mit MBAP-Header und PDU. Direkter Ausschnitt aus Folie 8.

3. Secure Shell (SSH)

3.1 Aufgaben von SSH

SSH erstellt eine sichere Verbindung zwischen zwei Computersystemen. Der Foliensatz nennt drei zentrale Sicherheitsfunktionen:

- Verschlüsselung der gesamten Nutzlast und des SSH-Headers.
- Integritätsprüfung des gesamten Pakets, funktional ähnlich einer Prüfsumme, aber kryptographisch abgesichert.
- Authentifizierung des Client-Benutzers.

Typische Anwendungsfälle sind:

- „Sicheres Telnet“: Zugriff auf einen entfernten Rechner über eine lokale Command Shell, sofern der Benutzer die nötige Berechtigung besitzt.
- Tunneling ansonsten unsicherer TCP/IP-Verbindungen. Dabei kann Port Forwarding zu einem Zielrechner verwendet werden.

Sachliche Ergänzung: Vertraulichkeit verhindert das Mitlesen des Inhalts. Integrität erkennt unbemerkte Veränderungen. Authentifizierung prüft, ob Server und Benutzer die behauptete Identität besitzen. Diese drei Funktionen werden in SSH getrennt ausgehandelt, wirken aber gemeinsam innerhalb derselben Sitzung.

3.2 Telnet

- Eine Command Shell stellt eine textuelle Benutzerschnittstelle zur Interaktion mit dem Betriebssystem bereit. Unter Windows ist cmd.exe ein Beispiel.
- Telnet öffnet eine TCP-Verbindung zu einem entfernten Computer und leitet Ein- und Ausgaben zwischen lokalem Terminal und entferntem System weiter.
- Der Benutzer muss sich am entfernten Computer anmelden.
- Alle Telnet-Daten einschließlich Benutzername und Passwort werden im Klartext übertragen.

Folgerung: Telnet bietet keine kryptographische Vertraulichkeit und keine sichere Integritätsprüfung. Es sollte deshalb nicht über nicht vertrauenswürdige Netze eingesetzt werden.

3.3 Funktionsweise einer SSH-Sitzung

Sitzungsverschlüsselung

- Zuerst wird eine normale TCP-Verbindung aufgebaut.
- Danach erfolgt ein Schlüsselaustausch mit asymmetrischen Public-Key-Verfahren. Der Foliensatz nennt das Diffie-Hellman-Verfahren.
- Die restliche Sitzung wird laut Foliensatz standardmäßig mit AES und 128-Bit-Schlüssellänge verschlüsselt.

Authentifizierung

- Der Benutzer kann sich mit User Credentials, also Benutzername und Passwort, anmelden.
- Alternativ kann die Anmeldung mit einem benutzerspezifischen Public Key erfolgen.
- Der Benutzer bleibt für die restliche Verbindung angemeldet. Die Verbindung ist damit zustandsbehaftet beziehungsweise stateful.

Sachliche Ergänzung: Beim Schlüsselaustausch wird ein gemeinsamer Sitzungsschlüssel erzeugt, ohne ihn direkt über das Netz zu übertragen. Die Server-Host-Schlüssel dienen zusätzlich dazu, den Server zu authentifizieren. Welche symmetrische Verschlüsselung tatsächlich verwendet wird, wird zwischen Client und Server ausgehandelt; AES-128 ist eine mögliche Variante. Moderne SSH-Implementierungen können auch andere Chiffren und authentifizierte Verschlüsselungsverfahren verwenden.

3.4 SSH-Tunnelling

Grundidee

- Nutzdaten, zum Beispiel ein HTTP-Request eines Clients, sollen durch einen verschlüsselten Tunnel geleitet werden.
- Die Nutzdaten werden verschlüsselt und anschließend beziehungsweise im Rahmen der SSH-Übertragung enkapsuliert.
- Beim Aufbau des Tunnels erfolgt ein Schlüsselaustausch zwischen Tunnelstart und Tunnelende. Das entspricht dem Aufbau der zuvor beschriebenen sicheren Remote-Shell-Sitzung.

Ziele

- Absicherung der Kommunikation über unsichere Netzbereiche hinweg.
- Umgehung von Firewalls mithilfe von Port Forwarding.

Sicherheits- und Betriebsaspekt: Port Forwarding darf nur in Netzen eingesetzt werden, in denen der Benutzer dazu berechtigt ist. Eine Firewall kann bewusst bestimmte Verbindungen blockieren; ein SSH-Tunnel kann diese Kontrolle umgehen und muss deshalb organisatorisch und technisch geregelt werden.

3.5 Struktur nach der Enkapsulierung

Die ursprünglichen Nutzdaten werden in eine SSH-Nachricht, also eine Protokolldateneinheit der Anwendungsschicht (L7-PDU), eingebettet:

- SSH-Header: enthält die Paketgröße der vollständigen Nachricht und die Größe des Padding-Feldes.
- SSH-Payload: enthält die ursprünglichen Nutzdaten.
- Padding-Feld: enthält eine variable Anzahl von Fülldaten.
- SSH-Trailer: enthält einen Message Authentication Code (MAC) zum Nachweis der Integrität.
- Die SSH-Nachricht wird als Ganzes verschlüsselt und mit TCP transportiert.

Die Foliengrafik zeigt außen IP und TCP, danach SSH-Header, das innere ursprüngliche Paket mit IP, TCP, HTTP und Daten, anschließend Padding und SSH-Trailer.

Sachliche Ergänzung: Die äußeren IP- und TCP-Header müssen für Router und den TCP-Transport lesbar bleiben. Verschlüsselt wird der SSH-Paketinhalt. Der konkrete Schutz von Längenfeldern, Padding und Integritätsdaten hängt von der ausgehandelten SSH-Chiffre und dem MAC- beziehungsweise AEAD-Verfahren ab.

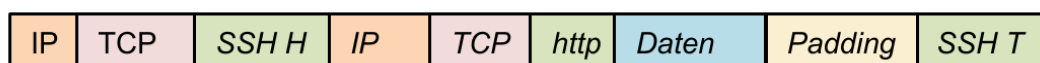


Abbildung 5: Struktur eines IP-Pakets im SSH-Tunnel. Direkter Ausschnitt aus Folie 13.

3.6 Port Forwarding

- Ist der Tunnel aufgebaut, horcht das Tunnelende auf Port 22 auf SSH-Daten.
- Der Tunnelstart horcht auf einem frei festlegbaren lokalen Port, im Beispiel auf Port 9023, auf Daten des Clients.
- Das Tunnelende muss wissen, an welchen Zielrechner und an welchen Zielport die dekapsulierten Daten weitergeleitet werden sollen.

Im gezeigten Beispiel verbindet sich ein Web-Client mit Port 9023 am Tunnelstart. Die Daten werden im SSH-Tunnel über Port 22 zum Tunnelende übertragen und von dort an Port 80 des Webserver weitergeleitet.

Sachliche Ergänzung: Dies ist lokales Port Forwarding: Eine lokale Adresse und ein lokaler Port werden auf eine Zieladresse und einen Zielport hinter dem SSH-Server abgebildet. Die Anwendung auf dem Client muss die SSH-Details nicht kennen; sie verbindet sich nur mit dem lokalen Weiterleitungsport.

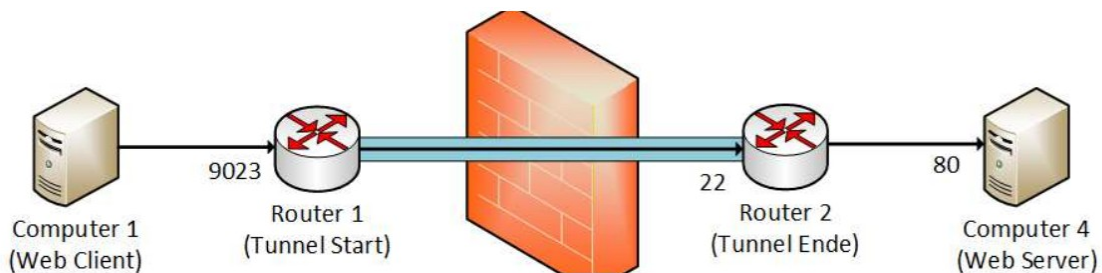


Abbildung 6: Schema des Port Forwarding mit lokalem Port 9023, SSH-Port 22 und HTTP-Zielport 80. Direkter Ausschnitt aus Folie 14.

4. Virtual Private Networks (VPN)

4.1 Grundidee von VPNs

- Ein VPN verwendet grundsätzlich einen Tunnel.
- Im dargestellten Fall gehören die beiden lokalen Netze organisatorisch zusammen.
- Kommunikation ist in beide Richtungen möglich.
- Die Endsysteme können je nach Datenfluss Client und Server sein.
- Tunnelstart und Tunnelende werden zu VPN-Gateways, also VPN-fähigen Routern oder Sicherheitskomponenten.
- Die IP-Adressen auf beiden Seiten werden von derselben Organisation verwaltet.

Sachliche Ergänzung: Ein VPN bildet über ein fremdes oder öffentliches Transportnetz eine logisch zusammengehörige private Kommunikationsumgebung. Je nach VPN-Technik werden nur die Pakete gekapselt oder zusätzlich verschlüsselt und authentifiziert.

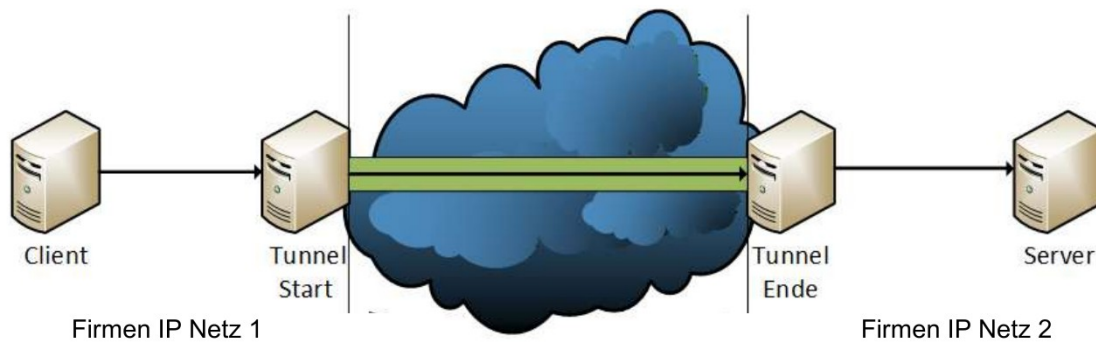


Abbildung 7: VPN-Tunnel zwischen zwei Corporate Networks. Direkter Ausschnitt aus Folie 15.

4.2 Site-to-Site-VPNs

- Site-to-Site-VPNs verbinden ganze Netzwerke oder Gruppen von Netzwerken eines Betreibers beziehungsweise einer Organisation.
- Endsysteme senden ihren Traffic an ein VPN-Gateway. Dort werden die Pakete verschlüsselt und enkapsuliert.
- Die verschlüsselten Daten werden zu einem anderen VPN-Gateway geleitet, hinter dem sich das Zielsystem befindet.
- Das Ziel-VPN-Gateway entschlüsselt und dekapsuliert die Pakete und leitet das ursprüngliche Paket zum Zielsystem weiter.

Sachliche Ergänzung: Die Endsysteme benötigen bei einem klassischen Site-to-Site-VPN normalerweise keine eigene VPN-Software. Die Gateways übernehmen Aufbau, Schlüsselverwaltung, Kapselung, Entschlüsselung und Routing.

4.3 Remote-Access-VPN

- Eine Seite des VPN reduziert sich auf einen einzelnen Rechner, den Client-PC.
- Auf dem Client-PC muss VPN-Client-Software installiert und konfiguriert sein.
- Der VPN-Client stellt eine Verbindung zum VPN-Server her.
- Der Client-PC erhält ein virtuelles Netzwerk-Interface. Dessen IP-Adresse wird vom VPN-Server vorgegeben.

- Der Client-PC verhält sich danach wie ein PC im Firmennetz.
- Die VPN-Software verschlüsselt und enkapsuliert die Daten und sendet sie an ein VPN-Gateway.
- Das VPN-Gateway verarbeitet die Daten anschließend wie bei einem Site-to-Site-VPN.

Sachliche Ergänzung: Das virtuelle Interface besitzt eine eigene IP-Konfiguration und eigene Routen. Das Betriebssystem entscheidet anhand seiner Routingtabelle, welche Pakete an dieses Interface und damit in den VPN-Tunnel gesendet werden.

4.4 Mögliche Protokolle für VPNs

Für VPNs können verschiedene Protokolle auf unterschiedlichen Schichten verwendet werden. Reine Schicht-2-Protokolle sind nicht direkt über Router weiterleitbar und benötigen deshalb einen geeigneten Transportmechanismus.

- Point-to-Point Tunneling Protocol (PPTP)
- Layer 2 Forwarding Protocol (L2F)
- Layer 2 Tunneling Protocol (L2TP)
- Generic Routing Encapsulation (GRE)
- Multiprotocol Label Switching (MPLS) VPN
- Internet Protocol Security (IPsec)
- Secure Socket Layer (SSL) beziehungsweise Transport Layer Security (TLS)

PPTP: Ein älteres Tunnelprotokoll für Punkt-zu-Punkt-Verbindungen. Die damit häufig verwendeten Sicherheitsmechanismen gelten heute als unzureichend.

L2F: Ein historisches Cisco-Protokoll zur Weiterleitung von Schicht-2-Sitzungen. Es wurde weitgehend durch L2TP abgelöst.

L2TP: Transportiert Schicht-2-Sitzungen über IP. L2TP selbst verschlüsselt nicht und wird deshalb häufig mit IPsec kombiniert.

GRE: Kann unterschiedliche Netzwerkprotokolle in IP enkapseln. GRE stellt allein keine Verschlüsselung und keine kryptographische Authentifizierung bereit.

MPLS-VPN: Trennt Kundennetze logisch im Netz eines Providers. Die Trennung basiert auf Routing- und Label-Mechanismen; eine Ende-zu-Ende-Verschlüsselung ist nicht automatisch enthalten.

IPsec: Schützt IP-Pakete durch Authentifizierung, Integrität und optional Verschlüsselung. IPsec kann im Transportmodus oder im Tunnelmodus eingesetzt werden.

SSL/TLS: Schützt Verbindungen oberhalb von TCP. Der Begriff SSL wird historisch verwendet; moderne Systeme setzen TLS ein.

4.5 Schicht-2-VPNs

- Der Begriff „Schicht-2-Tunnel“ ist missverständlich, weil er eine direkte Enkapsulierung in Schicht-2-Frames suggeriert. Solche Frames könnten nicht über normale IP-Router geroutet werden.
- Die Lösung besteht darin, zuerst einen Tunnel auf einer höheren Schicht aufzubauen. Innerhalb dieses Tunnels werden Frames zwischen zwei Standorten transportiert, die logisch zum selben IP-Netz gehören.

Sachliche Ergänzung: Ein Schicht-2-VPN erweitert eine gemeinsame Ethernet-Domäne über ein Transportnetz. Dadurch können Broadcasts, MAC-Adressen und gegebenenfalls VLAN-Informationen standortübergreifend übertragen werden. Der äußere Tunnel sorgt dafür, dass die inneren Ethernet-Frames über ein geroutetes Netz transportiert werden können.

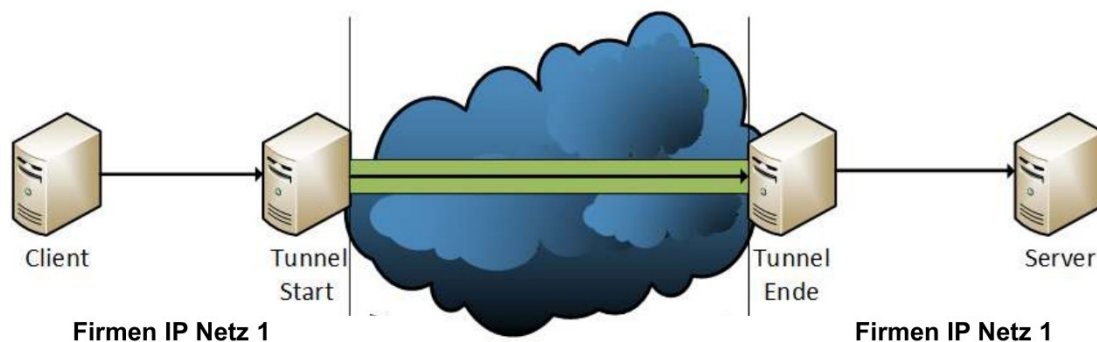


Abbildung 8: Layer-2-Tunnel zwischen zwei Teilen desselben IP-Netztes. Direkter Ausschnitt aus Folie 19.

4.6 SSL/TLS-VPNs

- SSL-VPNs basieren auf SSL/TLS-Tunneln.
- Clients stellen dabei häufig kein eigenes Netzwerk-Interface bereit. Die PCs behalten für den Zugriff auf das Firmennetz ihre bisherige IP-Adresse.
- Damit ist im Allgemeinen kein vollständiger Zugriff auf das gesamte Firmennetz möglich.
- Das VPN-Gateway muss die Anfrage an den angefragten Firmenserver weiterleiten.
- Der Firmenserver muss Anfragen aus fremden Netzen akzeptieren.
- Interne Firewalls und Router müssen diesen Traffic durchlassen.
- Auch die Antwort muss wieder korrekt zum Client zugestellt werden können.

Sachliche Ergänzung: Viele SSL/TLS-VPNs arbeiten als Webportal oder als anwendungsbezogener Proxy. Dadurch werden nur freigegebene Anwendungen veröffentlicht. Andere Varianten installieren einen Client und können ein virtuelles Interface bereitstellen; dann ist auch umfassenderer Netzwerkzugriff möglich.

4.7 Split Tunnel Routing

- Beim Split Tunneling werden nur kritische oder unternehmensbezogene Daten durch den VPN-Tunnel abgesichert.
- Der normale Internetzugang bleibt gleichzeitig direkt möglich.
- Das kann die Effizienz erhöhen, weil nicht der gesamte Internetverkehr durch das Unternehmensnetz geleitet werden muss.
- Das Verfahren ist riskant, weil korrekt entschieden werden muss, welcher Traffic durch den VPN-Tunnel und welcher direkt ins Internet gesendet wird.

Die Foliengrafik vergleicht eine Clientanbindung ohne Split Tunnel und mit Split Tunnel. Ohne Split Tunnel wird der Internetverkehr über VPN-Server und Unternehmensinfrastruktur geleitet. Mit Split Tunnel geht Unternehmensverkehr durch das VPN, während anderer Internetverkehr direkt über den lokalen Internetzugang läuft.

Sachliche Ergänzung: Fehlerhafte Routen oder DNS-Einstellungen können Daten am Tunnel vorbeileiten. Außerdem kann ein gleichzeitig mit dem Internet und dem Firmennetz verbundener Client eine zusätzliche Angriffsfläche bilden. Ein Full-Tunnel erleichtert zentrale Kontrolle, erhöht aber die Last auf VPN-Gateway und Unternehmensanbindung.

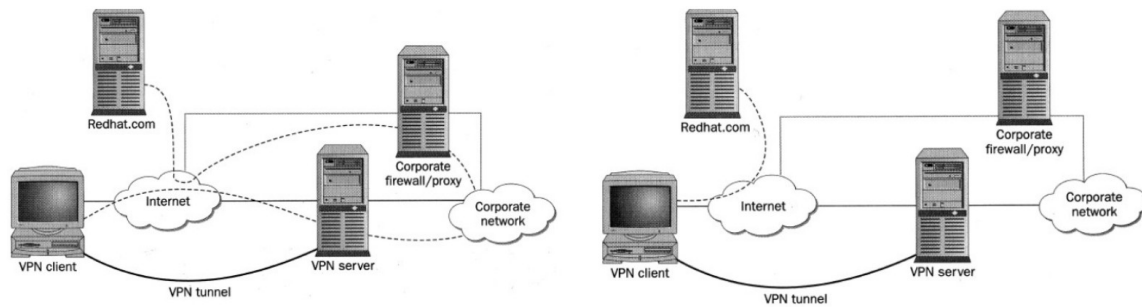


Abbildung 9: Clientanbindung ohne und mit Split Tunnel. Direkter Ausschnitt aus Folie 21.

5. Zusammenfassung der Kernbegriffe

- Tunneling: Transport eines vollständigen Protokolldatenelements als Nutzlast eines anderen Protokolls.
- Enkapsulierung: Hinzufügen äußerer Header und gegebenenfalls Trailer am Tunnelstart.
- Dekapsulierung: Entfernen der äußeren Protokollinformationen am Tunnelende.
- Verschlüsselung: Schutz des Inhalts vor unberechtigtem Lesen. Tunneling allein bedeutet nicht automatisch Verschlüsselung.
- 6in4: Transport von IPv6-Paketen in IPv4-Paketen; beide Tunnelenden benötigen IPv4 und IPv6.
- SSH: Sichere, zustandsbehaftete Verbindung mit Schlüsselaustausch, Verschlüsselung, Integritätsprüfung und Benutzer-Authentifizierung.
- SSH Port Forwarding: Weiterleitung einer lokalen TCP-Verbindung durch einen SSH-Tunnel zu einem festgelegten Ziel und Zielport.
- Site-to-Site-VPN: Verbindung ganzer Netze über VPN-Gateways.
- Remote-Access-VPN: Verbindung eines einzelnen Clients mit einem privaten Netz, häufig über ein virtuelles Interface.
- Schicht-2-VPN: Transport von Ethernet-Frames über einen Tunnel auf einer höheren, routbaren Schicht.
- SSL/TLS-VPN: TLS-geschützter Zugriff, häufig anwendungsbezogen oder über ein Webportal.
- Split Tunnel: Nur ausgewählter Traffic nutzt das VPN; anderer Traffic verwendet den direkten Internetzugang.

Ende des Skripts.

Erstellt auf Basis des Foliensatzes „VO Computernetzwerke – Kapitel 4“, SS 2026, Armin Veichtlbauer.